

Лабораторная работа №13.
Программирование в командном
процессоре ОС UNIX. Ветвления и
ЦИКЛЫ

Дисциплина: Архитектура компьютеров и операционные системы

Игнатенко Денис Бенъяминович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	9
4.1	Подготовка рабочего каталога	9
4.2	Написание скрипта поиска с <code>getopts</code> и <code>grep</code>	9
4.3	Анализ кода завершения программы на С	13
4.4	Создание и удаление временных файлов	16
4.5	Архивирование файлов с помощью <code>tar</code> и <code>find</code>	19
5	Ответы на контрольные вопросы	24
6	Выводы	26
	Список литературы	27

Список иллюстраций

4.1	Создание каталога lab13 и проверка текущего пути	9
4.2	Содержимое файла input.txt	10
4.3	Выдача права на выполнение скрипта search.sh	12
4.4	Поиск без учёта регистра	12
4.5	Поиск с учётом регистра	12
4.6	Поиск с выводом номеров строк	13
4.7	Запись результата поиска в файл result.txt	13
4.8	Компиляция программы check_number.c	14
4.9	Работа run_check.sh для положительного числа	15
4.10	Работа run_check.sh для отрицательного числа	15
4.11	Работа run_check.sh для нулевого значения	16
4.12	Создание временных файлов	18
4.13	Удаление временных файлов	18
4.14	Повторное удаление несуществующих файлов	19
4.15	Подготовка тестовой директории testdir	19
4.16	Архивирование всех файлов директории	21
4.17	Архивирование только свежих файлов	22
4.18	Содержимое файла files_to_archive.txt	22
4.19	Итоговый список файлов лабораторной работы	23

Список таблиц

3.1	Ключи скрипта <code>search.sh</code>	8
3.2	Основные команды лабораторной работы	8

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX и получить практические навыки написания командных файлов с использованием ветвлений, циклов, анализа параметров командной строки, кодов завершения программ и архивирования файлов.

2 Задание

- Написать командный файл, использующий `getopts` и `grep` для поиска строк по шаблону.
- Написать программу на языке C, определяющую знак введённого числа, и shell-скрипт, анализирующий код её завершения через `$?`.
- Написать командный файл, создающий и удаляющий пронумерованные временные файлы.
- Написать командный файл, архивирующий все файлы указанной директории или только файлы, изменённые менее недели назад.

3 Теоретическое введение

Командный процессор UNIX позволяет автоматизировать типовые действия с помощью shell-скриптов. В таких скриптах используются позиционные параметры, условные конструкции, циклы и внешние утилиты операционной системы. Такой подход удобен для обработки файлов, проверки условий и объединения нескольких программ в единую последовательность действий [1,2].

Для разбора параметров командной строки в shell применяется `getopts`. Эта команда поочередно читает ключи запуска и позволяет передавать в скрипт режимы работы и значения аргументов. Для поиска текста в файлах применяется `grep`, который поддерживает поиск по шаблону, вывод номеров строк и учёт или игнорирование регистра символов.

Код завершения программы передаёт оболочке информацию о результате её работы. В shell этот код доступен через специальную переменную `$?`. Благодаря этому можно запускать внешнюю программу и затем выбирать дальнейшие действия в зависимости от результата её выполнения.

Для перебора повторяющихся действий в скриптах применяются циклы `while`, `for` и `until`. Для проверки наличия обычного файла используется конструкция `test -f` или её сокращённая форма `[ -f файл ]`. При работе с наборами файлов удобно использовать архиватор `tar`, а для отбора файлов по времени изменения - утилиту `find` [3].

Параметры разработанного в лабораторной работе скрипта поиска приведены в таблице 1.

Таблица 3.1: Ключи скрипта `search.sh`

Ключ	Назначение
<code>-i inputfile</code>	указать входной файл
<code>-o outputfile</code>	записать результат в выходной файл
<code>-p pattern</code>	задать шаблон поиска
<code>-C</code>	различать большие и малые буквы
<code>-n</code>	выводить номера строк

Использованные в лабораторной работе команды и их назначение сведены в таблицу 2.

Таблица 3.2: Основные команды лабораторной работы

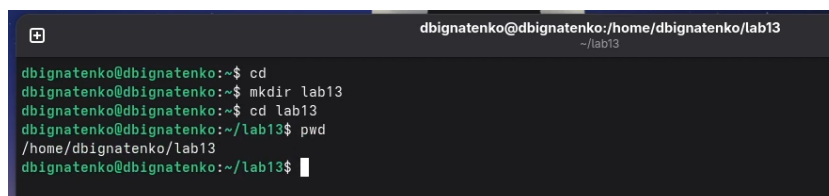
Команда	Назначение
<code>grep</code>	поиск строк по шаблону
<code>getopts</code>	разбор параметров командной строки
<code>gcc</code>	компиляция программы на языке C
<code>touch</code>	создание файла или изменение времени модификации
<code>rm</code>	удаление файлов
<code>tar</code>	создание и просмотр архива
<code>find</code>	поиск файлов по заданным условиям

4 Выполнение лабораторной работы

4.1 Подготовка рабочего каталога

Сначала был создан отдельный каталог lab13, после чего проверено текущее расположение командой pwd (рис. 1).

```
cd
mkdir lab13
cd lab13
pwd
```

A screenshot of a terminal window with a dark background. The window title is "dbignatenko@dbignatenko:/home/dbignatenko/lab13". The terminal shows the following commands and output: "dbignatenko@dbignatenko:~\$ cd", "dbignatenko@dbignatenko:~\$ mkdir lab13", "dbignatenko@dbignatenko:~\$ cd lab13", "dbignatenko@dbignatenko:~/lab13\$ pwd", and the output "/home/dbignatenko/lab13". The prompt "dbignatenko@dbignatenko:~/lab13\$" is shown at the bottom with a cursor.

```
dbignatenko@dbignatenko:~$ cd
dbignatenko@dbignatenko:~$ mkdir lab13
dbignatenko@dbignatenko:~$ cd lab13
dbignatenko@dbignatenko:~/lab13$ pwd
/home/dbignatenko/lab13
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.1: Создание каталога lab13 и проверка текущего пути

4.2 Написание скрипта поиска с getopt и grep

Для проверки работы скрипта был подготовлен входной файл input.txt с несколькими строками текста (рис. 2).

```
Linux is good
linux is powerful
```

UNIX shell scripting
Shell scripting is useful
grep can search text
Grep can be case sensitive

```
dbignatenko@dbignatenko:~/lab13$ nano input.txt
dbignatenko@dbignatenko:~/lab13$ cat input.txt
Linux is good
linux is powerful
UNIX shell scripting
Shell scripting is useful
grep can search text
Grep can be case sensitive
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.2: Содержимое файла input.txt

После этого был создан исполняемый shell-скрипт `search.sh`, для которого выданы права на запуск (рис. 3).

```
#!/bin/bash
```

```
case_sensitive=false
```

```
show_numbers=false
```

```
inputfile=""
```

```
outputfile=""
```

```
pattern=""
```

```
while getopts "i:o:p:Cn" opt
```

```
do
```

```
    case "$opt" in
```

```
        i) inputfile="$OPTARG" ;;
```

```
        o) outputfile="$OPTARG" ;;
```

```
        p) pattern="$OPTARG" ;;
```

```

        C) case_sensitive=true ;;
        n) show_numbers=true ;;
        *) echo "Ошибка: неверный ключ"; exit 1 ;;
    esac
done

if [ -z "$inputfile" ] || [ -z "$pattern" ]
then
    echo "Использование: $0 -i inputfile -p pattern [-o outputfile] [-C] [-n]"
    exit 1
fi

grep_options=""

if [ "$case_sensitive" = false ]
then
    grep_options="$grep_options -i"
fi

if [ "$show_numbers" = true ]
then
    grep_options="$grep_options -n"
fi

if [ -n "$outputfile" ]
then
    grep $grep_options "$pattern" "$inputfile" > "$outputfile"
    echo "Результат записан в файл $outputfile"
else

```

```
grep $grep_options "$pattern" "$inputfile"
fi
```

```
dbignatenko@dbignatenko:~/lab13$ chmod +x search.sh
dbignatenko@dbignatenko:~/lab13$ ls -l search.sh
-rwxr-xr-x. 1 dbignatenko dbignatenko 983 апр 28 00:13 search.sh
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.3: Выдача права на выполнение скрипта `search.sh`

Запуск без ключа `-C` показал, что поиск производится без учёта регистра: были найдены строки и с `Linux`, и с `linux` (рис. 4).

```
./search.sh -i input.txt -p linux
```

```
dbignatenko@dbignatenko:~/lab13$ ./search.sh -i input.txt -p linux
linux is good
linux is powerful
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.4: Поиск без учёта регистра

Затем выполнен запуск с ключом `-C`, при котором была найдена только строка, полностью совпадающая по регистру с шаблоном (рис. 5).

```
./search.sh -i input.txt -p linux -C
```

```
dbignatenko@dbignatenko:~/lab13$ ./search.sh -i input.txt -p linux -C
linux is powerful
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.5: Поиск с учётом регистра

При использовании ключа `-n` скрипт выводит номера строк, содержащих найденное слово (рис. 6).

```
./search.sh -i input.txt -p scripting -n
```

```
dbignatenko@dbignatenko:~/lab13$ ./search.sh -i input.txt -p scripting -n
3:UNIX shell scripting
4:Shell scripting is useful
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.6: Поиск с выводом номеров строк

В последнем тесте результат поиска был перенаправлен в файл `result.txt`, после чего содержимое этого файла было просмотрено (рис. 7).

```
./search.sh -i input.txt -p grep -o result.txt
cat result.txt
```

```
dbignatenko@dbignatenko:~/lab13$ ./search.sh -i input.txt -p grep -o result.txt
Результат записан в файл result.txt
dbignatenko@dbignatenko:~/lab13$ cat result.txt
grep can search text
Grep can be case sensitive
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.7: Запись результата поиска в файл `result.txt`

4.3 Анализ кода завершения программы на С

Следующим этапом была написана программа `check_number.c`, определяющая знак введённого числа и завершающаяся разными кодами в зависимости от результата.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int number;

    printf("Введите число: ");
    scanf("%d", &number);
```

```

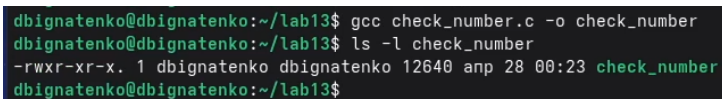
if (number > 0)
{
    exit(1);
}
else if (number < 0)
{
    exit(2);
}
else
{
    exit(0);
}
}

```

Программа была скомпилирована с помощью gcc, после чего проверено наличие исполняемого файла check_number (рис. 8).

```
gcc check_number.c -o check_number
```

```
ls -l check_number
```



```

dbignatenko@dbignatenko:~/lab13$ gcc check_number.c -o check_number
dbignatenko@dbignatenko:~/lab13$ ls -l check_number
-rwxr-xr-x. 1 dbignatenko dbignatenko 12640 apr 28 00:23 check_number
dbignatenko@dbignatenko:~/lab13$

```

Рис. 4.8: Компиляция программы check_number.c

Для анализа кода завершения был создан shell-скрипт run_check.sh.

```
#!/bin/bash
```

```
./check_number
```

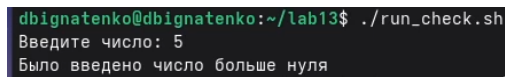
```

code=$?

if [ "$code" -eq 1 ]
then
    echo "Было введено число больше нуля"
elif [ "$code" -eq 2 ]
then
    echo "Было введено число меньше нуля"
elif [ "$code" -eq 0 ]
then
    echo "Было введено число, равное нулю"
else
    echo "Неизвестный код завершения: $code"
fi

```

Проверка скрипта выполнена для трёх случаев: положительного, отрицательного и нулевого значения. При вводе числа 5 скрипт сообщил, что введено число больше нуля (рис. 9).



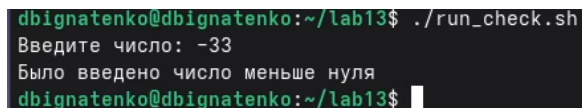
```

dbignatenko@dbignatenko:~/lab13$ ./run_check.sh
Введите число: 5
Было введено число больше нуля

```

Рис. 4.9: Работа run_check.sh для положительного числа

При вводе числа -3 было получено сообщение о вводе отрицательного числа (рис. 10).



```

dbignatenko@dbignatenko:~/lab13$ ./run_check.sh
Введите число: -3
Было введено число меньше нуля
dbignatenko@dbignatenko:~/lab13$

```

Рис. 4.10: Работа run_check.sh для отрицательного числа

При вводе числа 0 скрипт корректно определил нулевое значение (рис. 11).

```
dbignatenko@dbignatenko:~/lab13$ ./run_check.sh
Введите число: 0
Было введено число, равное нулю
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.11: Работа run_check.sh для нулевого значения

4.4 Создание и удаление временных файлов

На следующем этапе был разработан скрипт tmp_files.sh, работающий в двух режимах: create и delete.

```
#!/bin/bash

action="$1"
count="$2"

if [ -z "$action" ] || [ -z "$count" ]
then
    echo "Использование: $0 create|delete N"
    exit 1
fi

if [ "$count" -le 0 ]
then
    echo "Ошибка: N должно быть больше нуля"
    exit 1
fi

case "$action" in
    create)
        i=1
```



```

while [ "$i" -le "$count" ]
do
    touch "$i.tmp"
    echo "Создан файл $i.tmp"
    i=$((i + 1))
done
;;
delete)
i=1
while [ "$i" -le "$count" ]
do
    if [ -f "$i.tmp" ]
    then
        rm "$i.tmp"
        echo "Удалён файл $i.tmp"
    else
        echo "Файл $i.tmp не существует"
    fi
    i=$((i + 1))
done
;;
*)
    echo "Ошибка: используйте create или delete"
    exit 1
;;
esac

```

Сначала была проверена работа режима создания: скрипт создал пять файлов 1.tmp ... 5.tmp, что подтверждается выводом `ls -l *.tmp` (рис. 12).

```
./tmp_files.sh create 5
```

```
ls -l *.tmp
```

```
dbignatenko@dbignatenko:~/lab13$ ./tmp_files.sh create 5
Создан файл 1.tmp
Создан файл 2.tmp
Создан файл 3.tmp
Создан файл 4.tmp
Создан файл 5.tmp
dbignatenko@dbignatenko:~/lab13$ ls -l *.tmp
-rw-r--r--. 1 dbignatenko dbignatenko 0 апр 28 00:34 1.tmp
-rw-r--r--. 1 dbignatenko dbignatenko 0 апр 28 00:34 2.tmp
-rw-r--r--. 1 dbignatenko dbignatenko 0 апр 28 00:34 3.tmp
-rw-r--r--. 1 dbignatenko dbignatenko 0 апр 28 00:34 4.tmp
-rw-r--r--. 1 dbignatenko dbignatenko 0 апр 28 00:34 5.tmp
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.12: Создание временных файлов

Затем скрипт был запущен в режиме удаления. Все ранее созданные файлы были удалены, а команда `ls -l *.tmp` сообщила об их отсутствии (рис. 13).

```
./tmp_files.sh delete 5
```

```
ls -l *.tmp
```

```
dbignatenko@dbignatenko:~/lab13$ ./tmp_files.sh delete 5
Удалён файл 1.tmp
Удалён файл 2.tmp
Удалён файл 3.tmp
Удалён файл 4.tmp
Удалён файл 5.tmp
dbignatenko@dbignatenko:~/lab13$ ls -l *.tmp
ls: невозможно получить доступ к '*.tmp': Нет такого файла или каталога
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.13: Удаление временных файлов

Повторный запуск в режиме удаления показал корректную проверку существования файлов: скрипт выводит сообщение для каждого отсутствующего файла (рис. 14).

```
./tmp_files.sh delete 5
```

```
dbignatenko@dbignatenko:~/lab13$ ./tmp_files.sh delete 5
Файл 1.tmp не существует
Файл 2.tmp не существует
Файл 3.tmp не существует
Файл 4.tmp не существует
Файл 5.tmp не существует
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.14: Повторное удаление несуществующих файлов

4.5 Архивирование файлов с помощью tar и find

Для последнего задания была подготовлена тестовая директория `testdir`, содержащая файлы `old.txt`, `new1.txt` и `new2.txt`. Для файла `old.txt` была искусственно установлена дата изменения на десять дней раньше, чтобы он не попал в архив свежих файлов (рис. 15).

```
mkdir testdir
echo "old file" > testdir/old.txt
echo "new file 1" > testdir/new1.txt
echo "new file 2" > testdir/new2.txt
touch -d "10 days ago" testdir/old.txt
ls -l testdir
```

```
dbignatenko@dbignatenko:~/lab13$ mkdir testdir
dbignatenko@dbignatenko:~/lab13$ echo "old file" > testdir/old.txt
dbignatenko@dbignatenko:~/lab13$ echo "new file 1" > testdir/new1.txt
dbignatenko@dbignatenko:~/lab13$ echo "new file 2" > testdir/new2.txt
dbignatenko@dbignatenko:~/lab13$ touch -d "10 days ago" testdir/old.txt
dbignatenko@dbignatenko:~/lab13$ ls -l testdir
итого 12
-rw-r--r--. 1 dbignatenko dbignatenko 11 апр 28 00:37 new1.txt
-rw-r--r--. 1 dbignatenko dbignatenko 11 апр 28 00:37 new2.txt
-rw-r--r--. 1 dbignatenko dbignatenko  9 апр 18 00:37 old.txt
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.15: Подготовка тестовой директории `testdir`

После этого был написан скрипт `archive.sh`, поддерживающий два режима работы: `all` и `week`.

```
#!/bin/bash

mode="$1"
directory="$2"
archive="$3"

if [ -z "$mode" ] || [ -z "$directory" ] || [ -z "$archive" ]
then
    echo "Использование: $0 all|week directory archive.tar"
    exit 1
fi

if [ ! -d "$directory" ]
then
    echo "Ошибка: директория $directory не существует"
    exit 1
fi

case "$mode" in
    all)
        tar -cvf "$archive" -C "$directory" .
        echo "Архив $archive создан из всех файлов директории $directory"
        ;;
    week)
        find "$directory" -type f -mtime -7 -print > files_to_archive.txt

        if [ ! -s files_to_archive.txt ]
        then
            echo "Нет файлов, изменённых менее недели назад"
        fi
    esac

```

```

        exit 1
    fi

    tar -cvf "$archive" -T files_to_archive.txt
    echo "Архив $archive создан из файлов, изменённых менее недели назад"
    ;;
*)
    echo "Ошибка: используйте режим all или week"
    exit 1
    ;;
esac

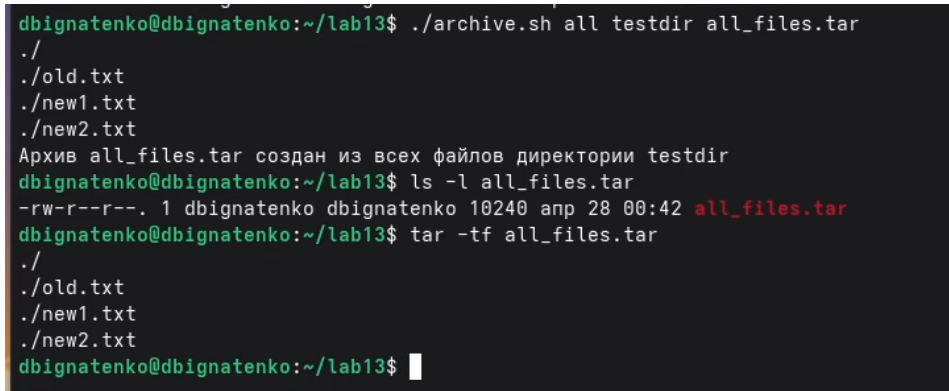
```

В режиме all скрипт создал архив `all_files.tar`, содержащий все файлы из директории `testdir` (рис. 16).

```

./archive.sh all testdir all_files.tar
ls -l all_files.tar
tar -tf all_files.tar

```



```

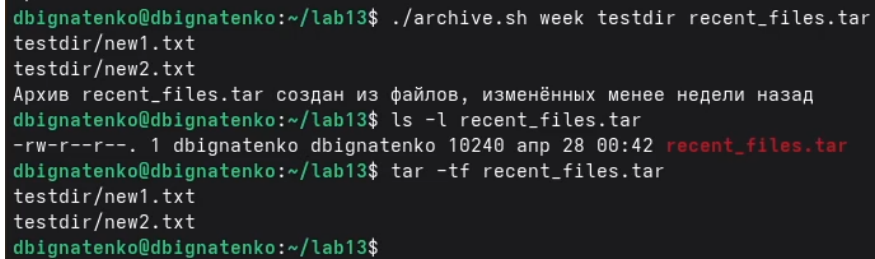
dbignatenko@dbignatenko:~/lab13$ ./archive.sh all testdir all_files.tar
./
./old.txt
./new1.txt
./new2.txt
Архив all_files.tar создан из всех файлов директории testdir
dbignatenko@dbignatenko:~/lab13$ ls -l all_files.tar
-rw-r--r--. 1 dbignatenko dbignatenko 10240 anp 28 00:42 all_files.tar
dbignatenko@dbignatenko:~/lab13$ tar -tf all_files.tar
./
./old.txt
./new1.txt
./new2.txt
dbignatenko@dbignatenko:~/lab13$

```

Рис. 4.16: Архивирование всех файлов директории

В режиме week в архив `recent_files.tar` были включены только файлы `new1.txt` и `new2.txt`, изменённые менее недели назад. Файл `old.txt` в архив не вошёл (рис. 17).

```
./archive.sh week testdir recent_files.tar  
ls -l recent_files.tar  
tar -tf recent_files.tar
```

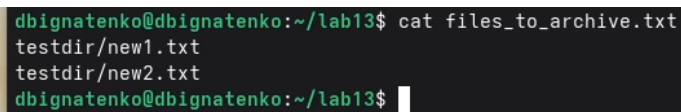


```
dbignatenko@dbignatenko:~/lab13$ ./archive.sh week testdir recent_files.tar  
testdir/new1.txt  
testdir/new2.txt  
Архив recent_files.tar создан из файлов, изменённых менее недели назад  
dbignatenko@dbignatenko:~/lab13$ ls -l recent_files.tar  
-rw-r--r--. 1 dbignatenko dbignatenko 10240 anp 28 00:42 recent_files.tar  
dbignatenko@dbignatenko:~/lab13$ tar -tf recent_files.tar  
testdir/new1.txt  
testdir/new2.txt  
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.17: Архивирование только свежих файлов

Дополнительно был просмотрен файл `files_to_archive.txt`, сформированный командой `find` и содержащий список файлов для архивации (рис. 18).

```
cat files_to_archive.txt
```



```
dbignatenko@dbignatenko:~/lab13$ cat files_to_archive.txt  
testdir/new1.txt  
testdir/new2.txt  
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.18: Содержимое файла `files_to_archive.txt`

В завершение была выполнена итоговая проверка содержимого рабочего каталога, где видны все созданные в ходе лабораторной работы файлы: скрипты, программа на C, архивы, входные и вспомогательные файлы (рис. 19).

```
ls -l
```

```
dbignatenko@dbignatenko:/home/dbignatenko/lab13
~/lab13

dbignatenko@dbignatenko:~/lab13$ ls -l
итого 72
-rw-r--r--. 1 dbignatenko dbignatenko 10240 anp 28 00:42 all_files.tar
-rwxr-xr-x. 1 dbignatenko dbignatenko 1093 anp 28 00:41 archive.sh
-rwxr-xr-x. 1 dbignatenko dbignatenko 12640 anp 28 00:23 check_number
-rw-r--r--. 1 dbignatenko dbignatenko 283 anp 28 00:22 check_number.c
-rw-r--r--. 1 dbignatenko dbignatenko 34 anp 28 00:42 files_to_archive.txt
-rw-r--r--. 1 dbignatenko dbignatenko 127 anp 28 00:16 input.txt
-rw-r--r--. 1 dbignatenko dbignatenko 10240 anp 28 00:42 recent_files.tar
-rw-r--r--. 1 dbignatenko dbignatenko 48 anp 28 00:18 result.txt
-rwxr-xr-x. 1 dbignatenko dbignatenko 402 anp 28 00:25 run_check.sh
-rwxr-xr-x. 1 dbignatenko dbignatenko 983 anp 28 00:13 search.sh
drwxr-xr-x. 1 dbignatenko dbignatenko 46 anp 28 00:37 testdir
-rwxr-xr-x. 1 dbignatenko dbignatenko 936 anp 28 00:34 tmp_files.sh
dbignatenko@dbignatenko:~/lab13$
```

Рис. 4.19: Итоговый список файлов лабораторной работы

5 Ответы на контрольные вопросы

1. Каково предназначение команды `getopts`?

Команда `getopts` используется в shell-скриптах для последовательного разбора ключей и аргументов командной строки. С её помощью удобно организовать поддержку нескольких режимов работы скрипта и проверку переданных параметров.

2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы оболочки применяются для шаблонной подстановки имён файлов. Например, `*.tmp` соответствует всем файлам с расширением `.tmp`, а `? .txt` - файлам, в имени которых перед `.txt` стоит ровно один символ.

3. Какие операторы управления действиями вы знаете?

К основным конструкциям управления в shell относятся `if ... then ... else ... fi`, `case ... in ... esac`, `for ... do ... done`, `while ... do ... done` и `until ... do ... done`.

4. Какие операторы используются для прерывания цикла?

Для управления выполнением цикла используются команды `break` и `continue`. Команда `break` завершает цикл полностью, а `continue` прерывает только текущую итерацию и переводит выполнение к следующей.

5. Для чего нужны команды `false` и `true`?

Команда `true` всегда завершается с кодом 0, а команда `false` - с ненулевым кодом завершения. Их используют в логических выражениях, тестовых примерах и при организации циклов.

6. Что означает строка `if test -f man${s}/${i}.${s}`, встреченная в командном файле?

Эта строка проверяет существование обычного файла с именем, составленным из значений

переменных `s` и `i`. Если такой файл существует и не является каталогом или специальным объектом, условие считается истинным, и тело `if` выполняется.

7. Объясните различия между конструкциями `while` и `until`.

Цикл `while` выполняется, пока проверяемое условие истинно. Цикл `until`, наоборот, выполняется до тех пор, пока условие ложно, и завершается, когда оно становится истинным. Таким образом, эти конструкции противоположны по логике проверки условия.

6 Выводы

В ходе выполнения лабораторной работы были изучены средства программирования в командном процессоре UNIX. Были разработаны shell-скрипты для поиска строк по шаблону, обработки кодов завершения, создания и удаления временных файлов, а также архивирования данных. Кроме того, были закреплены практические навыки работы с командами `getopts`, `grep`, `tar`, `find` и базовыми управляющими конструкциями shell.

Список литературы

1. Newham C. Learning the bash Shell: Unix Shell Programming. O'Reilly Media, 2005. 354 с.
2. Robbins A. Bash Pocket Reference. O'Reilly Media, 2016. 156 с.
3. Zarrelli G. Mastering Bash. Packt Publishing, 2017. 502 с.